



What a great message—make your regulatory requirements part of your work, constantly thinking about how to make them as simple and effective as possible.

Testing in regulated environments is an opportunity to grow our T-shaped skills by collaborating with programmers and other team members, as you will see in the next story.

Collaborating to Test Medical Device Apps

*In Chapter 20, “Agile Testing for Mobile and Embedded Software,” **JeanAnn Harrison** recounted her first experience both testing mobile software and working in an agile environment. She subsequently joined a team that develops medical devices and again found benefits in collaborating with the whole team, especially when it came to meeting regulatory needs.*

I was hired by a medical device company to help bolster the current testing team, provide support to that team, and do some mentoring in the software development life cycle. The medical device domain was a brand-new experience, but my experience with testing mobile device software came in handy.

As I started out checking tests previously written by an outsourced group, it became clear the tests were only functional tests, with little regard for performance, stress, boundary condition, and usability tests. Hardware and operating system conditions were not considered in the requirements, which meant we lacked the minimum test coverage for a medical device. Not only were our users our customers, but they were patients wearing a monitor to provide heart data to their physician.

The difference between our medical device’s software and a tablet or phone application was how many smaller applications were needed for an end-to-end system to reach its conclusion. It was a highly complex testing project that helped prepare me for single mobile app testing projects, and it shone a light on understanding how hardware conditions and operating systems integrate with the mobile app. Considering tests beyond the functional user interface (UI) is vital to planning any mobile testing project.

I needed to learn more about how the files were created, processed, and then transmitted, as well as the high-level code design. I needed to learn the architecture of the entire system to better understand how each application interacted with another application. The interdependencies were very complex, making test cases more complex. In order to provide solid test coverage, I needed to spend time with the

development team and ask a lot of questions, sit in code reviews, and review log files to better understand where those interdependencies existed.

By spending time with the development team, I found our shared sense of humor helped us bond. My development team became comfortable working with me and were willing to show me what they wanted me to learn from them. We all recognized how valuable my observations could be to their design. One of the biggest problems with mobile software development is the lack of requirements. You don't know what you don't know. Benchmarks in particular tend to be extraordinarily vague. Exploratory testing can help by providing information for developers to more accurately design. Testers can use the opportunity to further learn the architecture when they are directly involved in the design process.

For example, while performing stress testing, I recognized I could, in a short period of time, see how the device's software would react to the added load, and I uncovered a major long-standing bug. Working together, the programmer and I discovered that the bug occurred more often than we would have thought possible. The bug had to be addressed immediately, so the programmer and I paired to solve the issue. In the end, I learned not only more about the software design, but also how to be more accurate in my testing. In mobile testing, working directly with the development team helps the tester to be educated in the architecture, which leads to a wider range of test coverage beyond functional and UI testing.

With medical devices such as the mobile heart monitor, the delivery team is required to provide documentation that could be traced to the requirements directly. Many people feel this traceability means matching up a specific requirement directly to a specific test case. However, not all requirements were written or submitted to be tested directly. Instead, the team designed use cases to prove to the auditor that the device and software worked as expected. These use cases provided a trace to design documents where I could develop more use case tests. The process was less intrusive for the programmer, and the testers submitted the necessary test documentation to satisfy the regulatory stakeholders.

Testing a mobile medical device has its challenges, especially when the device is a proprietary one. However, all mobile testers need to work closely with their development team to learn as much as they can about the architecture. Timing, various hardware conditions, the firmware dependencies, and the sequence of how the code is executed are all critical components when designing test cases. Working with the development team as a whole made our agile software project on the mobile heart monitor a fairly successful one.

JeanAnn's team found a lightweight solution for traceability of requirements that was acceptable to their regulatory auditors. There's a perception that regulated always means heavyweight, but in our experience, auditors are usually open to simple alternatives to meet their information requirements. Experiment and collaborate to find an optimal solution.

SUMMARY

Many aspects of agile development, such as short feedback loops and automated regression tests to provide living documentation, make agile practices suitable, even advantageous, in regulated environments. Bringing the subject matter experts together with testers and other development team members allows us to guide coding and mitigate risk with a wide range of examples. Regulated environments provide unique opportunities for testers who enjoy mastering business rules and helping the team deliver the right thing. Some of the aspects of agile approaches to testing in regulatory environments we covered in this chapter are:

- Contrary to popular belief, agile does not mean no documentation. Teams that use examples to guide development turn those into automated tests that provide living documentation that can help meet regulatory requirements.
- Complying with auditors' requirements for information doesn't necessarily mean producing old-school, heavyweight documents. Work with auditors to find lightweight ways to comply that fit with your team's needs.
- Government regulations add quality characteristics that need to be verified, even if they don't directly add business value. Avoiding fines and ensuring customer security are critical to the business.
- Tester-programmer collaboration can be essential for testing high-risk, regulated software products. Understanding the system design, hardware considerations, and timing issues helps in preventing defects or finding them early.

AGILE TESTING FOR DATA WAREHOUSES AND BUSINESS INTELLIGENCE SYSTEMS

22. Agile Testing for Data Warehouses
and Business Intelligence Systems

What Is Unique about Testing BI/DW?

Using Agile Principles

Data—the Critical Asset

Big Data

A data warehouse (DW) is an integrated environment that contains operational as well as decision support data. It also stores data that is created within the data warehousing environment such as aggregates, summaries, and calculations. It is an effective way of managing the decision support function in an organization by treating data like a strategic asset that can be used for competitive advantage. A good system provides easy access to relevant data, allowing modeling as well as retrospective and predictive analysis to better inform business decisions—hence the idea of business intelligence (BI). Modeling the enterprise for a data warehouse is a huge effort, and in traditional data warehouse development approaches that require the full data model to be defined first, it takes years for the business to see any benefit.

With agile methods, using fast feedback cycles, the business people can prioritize what information they want to see first. They generally have little or no idea of what they want until they see it, so again, the fast feedback cycle is beneficial to them. The idea of working incrementally means that data quality issues are exposed early, preventing massive issues later on.

WHAT IS UNIQUE ABOUT TESTING BI/DW?

Under the BI umbrella, you use the system as a consumer of the data—more of an end user type of role. For DW, you get down into the technical delivery and need to understand the architectural solution. Figure 22-1 is an example diagram of a BI/DW solution.

One huge challenge is finding testers with the necessary skills: technical knowledge to verify the data movement from source to target, technical knowledge of databases, testing skills in data integrity to ensure that the data is consistent, security testing knowledge, and analytical skills to understand what the data represents. Is it even reasonable to think these skills can be found in one person? It begins to sound like that “whole-team approach” again, doesn’t it? The whole team needs to encompass even more disciplines. It is important that application development consider the data warehouse, because data quality issues can hamper testing and completion of stories.

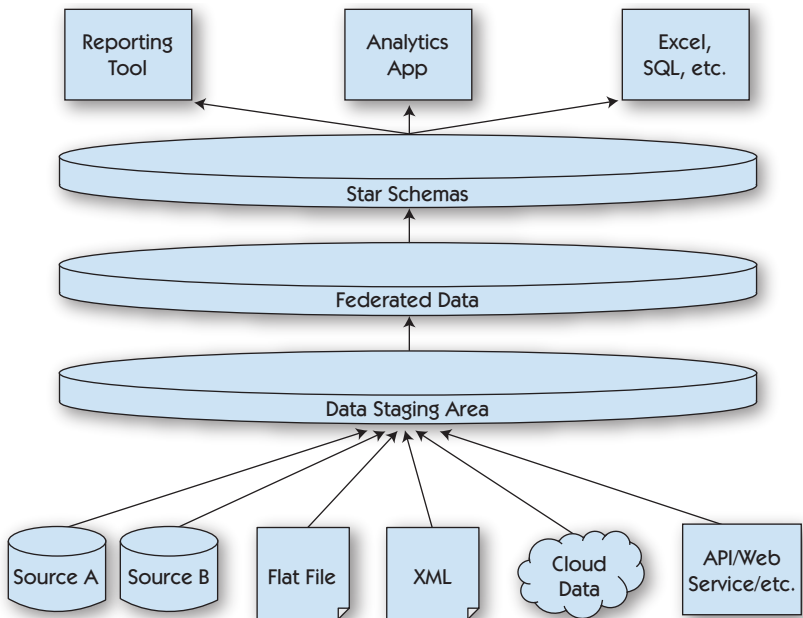


Figure 22-1 Example of a data warehouse setup (supplied by Mike Heinrich)

Teams implement agile practices differently with business intelligence and data warehouse features from the way they do in a regular web application. For example, there may be nothing visible to demo via a user interface for a long time. The value is in the data, which may be best demonstrated early using command-line tools. Data warehousing is based on a batch model, where the functionality is about moving high volumes of data in batches, rather than processing transactions continuously, so continuous integration (CI) is not as meaningful. This means that the regression tests might not be run as part of a daily build, but rather as part of the daily operational load processes. However, if automated tests with related test data are created for each story, many of the tests can be run in a CI environment.

Traditionally, DW/BI teams have used large volumes of data to test code. It often could take many hours or days to load all that data and see the test results. Agile teams are finding ways to either reduce the volume of data needed to adequately test stories or leverage new technologies that significantly speed up extract, transform, and load (ETL) processes to allow testing on large volumes of data. Our contributor stories in this chapter will show you some of the ways people have adapted.

Unit tests for DW/BI look different from application unit tests. A typical unit test might ensure that row counts and totals match from source to target, and these types of ETL tests can be automated. However, the transformation step changes the way data is represented. For example, data types may be converted, calculations applied, or daily values aggregated into monthly totals. These types of transformations represent some of the risks in data management that often make it harder to unit test.

To test DW/BI, an excellent understanding of data boundary values, ranges, and type transformations is essential. It's easy to fall into a trap of having many automated tests running that are so simplistic they don't add any value. Simple tests might be generated or copied easily since inputs are generally through a command-line interface. The difficulty is in understanding the different decision rules and paths that the data can take and executing those. Problems in data manipulation and transformation often occur at the boundaries between types. Understanding the nature of data types within database systems and the decision rules that are applied to the data as it is processed is needed for creating valid and informative tests.

Table 22-1 Simple Checklist for Automation

Test #	Description	Risk	Objective
1	Verify no records are lost	High	Records are transferred completely
2	Verify no fields are lost	High	Records are transferred completely
3	Verify source and target totals match	High	Records are transferred completely
4	Verify no data size errors	Medium	Records are processed correctly
5	Verify no data type errors	Low	Records are processed correctly
6	Verify data format changes are performed correctly	Low	Records are processed correctly

Collaborating with the programmers and product owners enables testers to target tests effectively. A combination of communication and exploration can help to identify the right tests to automate to reduce the risks of ineffective tests.

Automation is necessary, but it has a different appearance from what we discussed in Part VI, “Test Automation.” Often it is in the form of simple scripts that compare fields. A simple checklist may help identify some of the issues and highlight what can be automated for testing data warehouse applications. A colleague of Janet’s started with a simple checklist, similar to Table 22-1.

You may have to apply formatting or masking of data in order to perform a consistent check, storing the original state of the data at each stage of the test to help track where any problems have occurred. In addition, capturing all of the logs and traces from the process will help in diagnosing any problems later.



Try using an automated process to execute tests, creating and injecting data, and then examine the results and use exploratory testing to investigate further. Adam Knight’s teams use the automated harnesses to run their exploratory tests, since it is easier to manipulate data that way. Note that due to the large amounts of data involved, you may need to monitor activity in production to ensure that edge conditions are handled correctly.

USING AGILE PRINCIPLES

Trying different test approaches iteratively helps teams discover good ways to demonstrate data quality and the value it can add to the business. Basic agile principles such as simplicity, fast feedback, and breaking things into small chunks can be applied to BI/DW systems.

Learning to Test BI

Mike Heinrich *shares his experiences in learning to test in a data warehouse project. Trial and error ruled for the first little while, but it didn't take long for him to get into the rhythm.*

During the process of building our data warehouse and business intelligence environment, we learned very quickly that the business was not interested in the intermediate steps required to stage and transform the data through multiple databases. The statement, "But I can do this in Excel much faster" was a common refrain. With that in mind, it became very evident that we needed to show the business its usable, modeled information early and often. Of course, to stage all the data, federate it, and transform it into something the business could evaluate via traditional ETL processes is expensive, time-consuming, and change averse.

This led us to find ways to abstract the logical data models and to consider what the business cared about, rather than concentrating on the storage medium. Using our reporting tools to do rudimentary data federation and transformation shortened the feedback loop and allowed the business people to use a tool they were familiar with to refine their data requirements. By giving our business something tangible to explore, we reduced the number of data quality issues and even helped refine from where data should actually be sourced.

As the data requirements became better understood, the development team was able to determine whether ETL and data persistence were required or if the existing solution was satisfactory. My role as tester throughout all of this became less about verifying the functionality of ETL scripts and more about brokering the communication between the business and the development team around data quality and performance expectations.

As we moved into user acceptance testing (UAT), the iterative approach was once again necessary as we learned that the concept of a "rubber-stamp UAT," where the testing team had already discovered

and communicated all the issues, would be completely unsuccessful and guarantee we would never get to production. UAT became an intensive process for which we needed even more frequent communication between the business and the delivery team as they finally worked through their completed, overall model. Accepting that what we had called UAT was really just the final “dev and test” iteration of the product was perhaps the biggest mental hurdle we had to overcome.

The concept of model-driven development, or populating a loosely defined (and changing) logical model to allow the business to understand the data earlier, was not necessarily a popular approach. However, the shortened feedback loops allowed a project that had cost millions and languished in development for years to finally deliver valuable insight into customers and costs to support the profitability of the organization.

Mike used many agile principles in learning how to test his first BI/DW project. By understanding what the customer found valuable, the team was able to adapt its approach to meet those needs. Fast feedback loops gave the team the confidence to experiment to see what worked best. Janet was fortunate enough to talk with Mike during the project, and one of the many problems they had was bad data. The team suffered with an age-old problem—GIGO (garbage in, garbage out). With the fast feedback cycles, they were able to start cleaning up the data as they were developing, rather than having to wait until everything was built.

To ensure smooth iterative development, identify your test environment requirements at the start of each activity so that you are not blocked waiting for hardware and environments where you can execute your tests.

DATA—THE CRITICAL ASSET

Data warehouses are generally governed by strict rules. One example is that the data may not be volatile—it must never be overwritten or deleted. Once database transactions are committed, the data is static, read-only, and retained only for future reporting. This may involve huge amounts of unique data with a limited-value life cycle, which is why we hear so much these days about Big Data. Testing with so much data becomes an issue of its own, and we will cover that later in this chapter.

Another issue is privacy and security. It's not always possible to use production data for testing. Most countries have a privacy regulation protecting personal details. This means that production data needs to be "scrubbed" or "depersonalized." The database administrator on Lisa's previous team created a procedure using keys with a mathematical computation that replaced sensitive information such as IDs within the system. If needed, the IDs could be decrypted to identify the values in production. If a unidirectional approach is acceptable, you can simply use randomly generated numbers. Some database products have built-in procedures that can be used. Make sure that the substituted values meet the same constraints to which production data is subjected.

In business intelligence, the data is the most important asset. Since business decisions are based on that data and how it is represented, the way we handle the data for testing is critical.

Solving the Bad Test Data Problem

Lynn Winterboer, an agile trainer and consultant for BI/DW solutions, tells how her team solved one problem they faced when testing without appropriate test data.

All who work in the data warehousing and business intelligence industry understand how data quality can impact BI results. This is true for testing as well as production loads and can be particularly painful when the business rules being applied in the ETL are complex and overlapping. A BI team can save itself many headaches by starting out with a clean set of specific test records for unit and regression testing for the main scenarios of the project.

For example, I worked with one company that had grown over 20 years, largely by acquisition. When companies were acquired, their order data was integrated into the enterprise resource planning (ERP) system by people unfamiliar with data management. They were only concerned about getting the data to somehow fit in the target operational system so that the combined businesses could continue to operate. Unfortunately, this caused many data anomalies in the ERP system.

On a revenue allocation project that required order data from this ERP system, our DW/BI team worked to implement complicated business rules. To test the ETLs that applied these rules, we used random

samples of production data. We repeatedly experienced trouble due to the quality of our test data. Problems looked like this:

1. Code and test Product A.
2. Code and test Product B.
3. Regression test Product A with the same record we used to test it the first time, but this time, the test fails.
4. Working with the business lead on the project, we eventually figured out that the record we had used for Product A had an anomaly in it that caused a false negative on the regression test when the rules for Product B were also in the ETL. On a clean record for Product A, the test would have passed.

Because of these issues, we struggled just to code and successfully test our happy path scenarios, much less the corner cases. The team was frustrated.

The business lead decided to focus her time on creating a small, accurate set of test data that could be used to confirm that we had all our coding logic correct for the main scenarios in new orders. She identified 16 clean orders that we could use to test and regression test the ETL as each scenario was added, and then manually calculate the appropriate revenue allocation results for each.

This allowed our team to refocus on the most important deliverables. We could trust that when a test failed, it was due to a real code issue within a specific scenario related to the record that failed the test, and not bad test data. This saved lots of wasted time. Our business lead now had time to research the data anomalies and decide whether to fix them in the source, provide business rules to address them, or determine whether they should be included in the initial load. She was also able to clarify data input standards on the source system going forward, so we could avoid this problem with the next inevitable acquisition or data integration.



Can you spot some of the agile practices that Lynn used in her story? Guiding development with examples is the first one we noticed. It's not the normal format we think about, but the practice is the same. Provide real examples, and then code to make the tests pass. The second practice that stood out for us was the business lead creating the test data based on the most important scenarios. They used small experiments with fast feedback to evolve their testing and coding approach.

Managing Data for Tests

*In this next story, **Jeff “Cheezy” Morgan** shares a pattern that he found useful when he was trying to automate and populate test data.*

A few years ago I was working with a team that was developing a data warehousing application using ETL. The source was a very large database with about 140 tables, and the target was a large star schema database (Wikipedia, 2014n).

When I first arrived, the testers were testing the application by running about 500,000 records through the system each night. When they came to work the next morning, they ran queries against the source to find records that matched the condition they wished to test. Once they found an appropriate record, they looked for the same record in the target and interrogated the data to make sure the business rule was triggered and that it correctly modified the data. Needless to say, this process was extremely laborious and error-prone.

Right away I wanted to automate the process, but in order to begin the automation I needed a way to set up the data in the source. It was very intimidating to think about having each test insert data into so many tables. The complexity was going to be enormous, so I began looking for a way to simplify the test data management.

I soon discovered that each business rule performing the ETL process looked at only a small subset of the overall data passing through it to determine if it should trigger the rule or not. I realized that for each test, I could simply populate the tables with default values. Then for each specific test, I could replace only the few values that changed the outcome of a specific test with the values I wanted for that test. This dramatically simplified everything. It took me four days to create a framework that provided this ability. After that, each tester was able to supply specific values for a small subset of data needed to verify a particular test case and let the test use the default values for everything else. Figure 22-2 shows this setup.

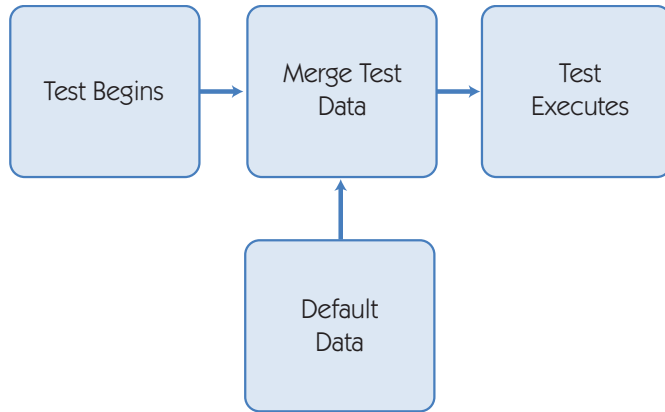


Figure 22-2 Changing selected default data for tests

The next project I worked on was a web application. I soon discovered that the same pattern applied. A lot of data was needed for each test workflow, but the amount of data that actually changed the outcome of the tests was a small subset. By specifying only the subset of data and allowing a framework to populate the remainder, I was able to simplify the tests.

The final discovery that I made was when I started executing the tests in parallel in order to get them to finish within a shorter amount of time. When you do this, you lose control over the order in which your tests run. If two tests running at the same time are trying to use the same data, it is likely that each will modify the data the other needs, and one or both of the tests will fail. To avoid this type of failure, you can use automation to randomize the data used by the tests so that each test has its own unique input values, and multiple tests can safely be run concurrently.

Data management can be painful. Writing scripts to iteratively create data sets may be preferable to extracting and scrubbing vast quantities of data from a production database. One approach is to iteratively build a data generator. Start with records that test the simplest rules, and then add support for different rules as you identify them. For example, null

values, orphan records, and high-cardinality records may provide useful tests for the integrity of the data transformations and reporting. Building a generator rather than manually creating records allows your team to scale your data once you are happy with your functional coverage.

BIG DATA

Big Data is a challenge for many organizations. The concepts of volume, velocity, and variety are really what define Big Data, and traditional data warehouse approaches may not scale sufficiently for testing. When performance and data volume are high-priority considerations, agile teams need creative testing solutions.

Testing Performance and Scale in Agile

*In testing a product that operates on a Big Data scale, **Adam Knight** addresses one of the biggest challenges for testing agile projects: ensuring that the software meets the performance and scale requirements within the short time-boxed iterations available for testing.*

One of the biggest challenges when testing a data warehouse or data processing system is ensuring that the system can meet the performance and scalability demands that are placed on it. Performance and scale are important quality characteristics of any application development. However, they are a particular priority for systems that manage large quantities of data.

This presents something of a challenge to agile teams. When working in small teams, testing sufficiently against performance targets within the short sprint iterations can appear to be a daunting task. In production environments, it can take many days or weeks to accumulate the volume of data that constitutes the operating capacity of Big Data systems. We rarely have the luxury of these timescales within agile sprints.

Scale at Speed

In the past, a brute-force approach to testing at scale may have been an option as part of a nonfunctional phase of a staged testing project. In an agile context, we need to be creative when testing these important characteristics at the required scale and at the speed needed for agile

development. Over the years of tackling exactly these types of problems, my team has come up with a number of approaches to help us:

- **Scaling layers:** In order to process large data volumes in parallel, large data systems typically incorporate scalability layers of metadata within them. These often take the form of metadata indexes or databases. If we can understand and control these layers, we can quickly generate installations that exhibit the same characteristics as production systems in terms of the metadata processing but require a fraction of the time and space to generate. Working with the programmers to understand the metadata structures of the application and how to manipulate these is an excellent approach to help in testing the scalability of the various system components in isolation.
- **Backup and restore:** By making use of our own backup and restore mechanisms, we can quickly generate environments in which we can test, whilst also creating extra confidence in our disaster recovery capabilities.
- **Static installations:** One effective approach is to maintain a number of environments that are populated with data at production scale against which we can schedule checks of key performance targets. In the simplest approach, we maintain a static system where we exercise new versions of our querying engines. In more advanced testing we maintain a rolling test in order to continuously cycle data in and out of a fully populated system. In this way we test the performance of the features at realistic scale without having to build the data up from scratch for every run of the tests.

Automating for Scale

As well as approaches to managing the software and data, there are a number of techniques within our automation harnesses that specifically help us target testing scalability and performance targets:

- **Iteration:** If we design our test packs carefully, our automation harnesses support the ability to run these iteratively. This allows us to scale up to large volumes of data loads and track the performance of other system operations. We can use iterative automation in exploratory testing to allow us to examine characteristics with increasing activity, or as part of ongoing testing such as running iterative tests against static installations as mentioned previously.
- **Parallelization:** Most data processing systems support parallel processing across multiple CPU cores or machines in order to

manage the volumes of data that are involved. It seems inevitable that our test automation must then also run in parallel if we are to exercise the application realistically. The ability to execute tests in parallel allows testers to drive realistic mixed workloads that deterministically exercise the multiple processes that we would expect to see when running a clustered application. It is sensible both to parallelize activity within a single test run and to parallelize automated test runs on different machines. This allows different workloads and types of tests to be run on different environments, optimizing the information that we obtain within an iteration. Smaller nightly test runs for the most important tests, combined with large-scale weekend testing across dozens of servers, allow us to develop quickly and with confidence. We also are able to maintain large-scale test suites that provide confidence in larger-scale characteristics.

- **Monitoring:** In addition to the outputs required to furnish checks, we capture information on the executing software and its environment whilst tests are executing. As we run tests iteratively and in parallel, we use the information gathered to graph system characteristics and model system behavior to identify potential scalability problems.

When working in an agile manner, it is easy to overlook testing of performance and system scalability. However, these are essential quality characteristics of data storage and analysis applications. By adopting a philosophy of making performance integral to our testing and using some creative ideas in both exploring and checking approaches, we can achieve a surprising amount of confidence in these characteristics, even within short iterations.

Big Data is a new and growing challenge for many teams and requires a diligent and multifaceted approach such as the one Adam describes. Performance testing in data warehousing is required to establish whether the application or system is capable of handling the ingestion of data and query workload. If, for example, the hardware or database software is not capable of processing the workload, attempts at performance tuning will not help. If the hardware and database can accommodate the scale of work, though, the team can take steps to tune the performance, measuring results and rerunning to evaluate results. Testing can help identify key scenarios, but having testers work in tandem with the programmers in an iterative manner pays off in ensuring data quality.

SUMMARY

Testing in a data warehouse or business intelligence system is a good example of a context that stretches the bounds of agile for a development team. In this chapter we shared ideas that we hope will help struggling teams.

- Consider the specialized skills your team will need to test a data warehouse or business intelligence system and fill gaps with training or bringing in people with those in-depth skills.
- In-depth technical skills, along with broad business domain knowledge, are needed to succeed with testing BI and DW systems.
- Agile principles and values such as testing early and using fast feedback apply to software that builds data to help with business decisions.
- To meet the challenge of verifying performance and scalability when dealing with huge amounts of data, look for creative ways to use metadata, parallelize automated test processing, and work in layers.
- Monitor system data in test and production to evaluate scalability and check for system response to edge cases.
- Remember privacy and security concerns when testing DW and BI systems.
- Data integrity is critical to business intelligence. If the data is bad, the decisions based on that data will be bad. Make sure the test data reflects real data.
- Keep in mind that scaling and performance need to be addressed from the beginning. Build your test strategy to scale with Big Data.

TESTING AND DEVOPS

	A Short Introduction to DevOps
23. Testing and DevOps	DevOps and Quality
	How Testers Add DevOps Value

We’ve worked for many years with team members who do activities that are now called DevOps. As Jez Humble says, team members engaged in DevOps build “a platform that allows developers to self-service environments for testing and production (and deployments to those environments)” (Humble, 2012). They provide tools that enable delivery teams to build, test, deploy, and run their systems. In our experience, everyone on agile teams is doing DevOps activities, but one or more team members contribute in-depth specialized skills. Giving the skill set a label has called attention to its importance in building quality into software. DevOps is an integral component of successful agile testing.

A SHORT INTRODUCTION TO DEVOPS

The term *DevOps* was first popularized by the DevOps Days conference in Belgium in 2009. Since then there have been some excellent publications devoted to DevOps and how it fits into agile development. *Continuous Delivery* (Humble and Farley, 2010) and *DevOps for Developers* (Hüttermann, 2012) are two good guides to DevOps.

DevOps includes practices and patterns that improve collaboration among different roles in delivery teams, streamlining the process of delivering high-quality software. These practices and patterns help teams write testable, maintainable code, package the product, deploy it, and support the deployed code.

One area of focus for DevOps is shortening cycle times. Some businesses are able to take this to an extreme, delivering new code to production

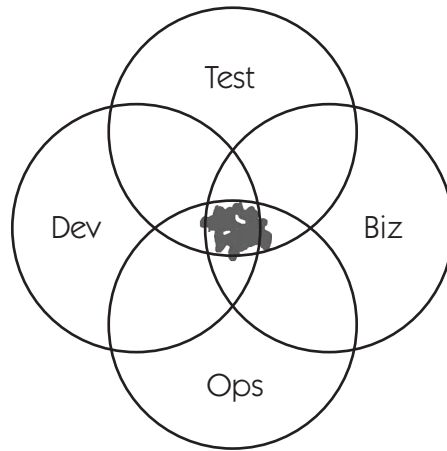


Figure 23-1 Intersection of development, testing, operations, and the business: the whole-team approach!

many times per day. Delivery team members find reliable, repeatable ways to get from an idea to delivered business value, with a cycle time that's not only short, but safe. This helps prevent defects and build in quality.

DevOps activities are designed to improve the maintainability and speed of automated tests and deployment. Examples include maintaining test environments, providing fast feedback from continuous integration (CI), and ensuring reliable and frequent—perhaps even continuous—deployment. Many teams now test their infrastructure continually to guard against spurious intermittent regression test failures due to server, deployment, or configuration failures. Business stakeholders are also involved, helping to hire the right people, obtain the necessary hardware and software, and gather information to guide future development. Figure 23-1 illustrates that DevOps is the intersection of programming, testing, operations, and the business. It is another way to look at the whole-team approach to agile testing.

DEVOPS AND QUALITY

Agile team members' skills often include scripting, system administration, coding, configuring CI, tooling, and collaboration and communication skills that enable them to perform DevOps activities. These activities are instrumental in agile testing and continually building quality into our products.

Janet's Story

Years ago, before the idea of continuous integration existed (at least in my world), I worked with a person I consider to be one of the first DevOps practitioners. Darcy's role included implementation of our product at client sites, first-level support for clients, keeping our servers running, as well as being our technical tester, setting up our test environments, and keeping them running. He was able to bring what he knew from the world of clients into our testing, including recovery testing. I did not realize the value he brought to the development of our product until I worked in an organization where that crossover role did not exist.

In another organization, we had an infrastructure test team. They worked with the programmers and the teams that supported the hardware and communications. They worked with each of the agile teams to help them understand the impacts of changes to the infrastructure on the new features.

People with specialized operations and system administration skills help the team improve quality on many different fronts. They can help set up appropriate development, test, and staging environments, optimize the CI, and refine the deployment processes for these environments. They can also help find or build and implement automation frameworks and other tools that suit the team's needs. In our experience, they're terrific at diagnosing problems and have many useful tools at their disposal. They can help with the infrastructure to support automated tests, such as generating test data.

Figure 23-2 illustrates the idea that DevOps crosses into all four of the agile testing quadrants. It helps guide development with many different types of testing activities, provides tools and environments for evaluating the product, and builds the technology that enables testing of diverse quality attributes such as performance, reliability, and security.

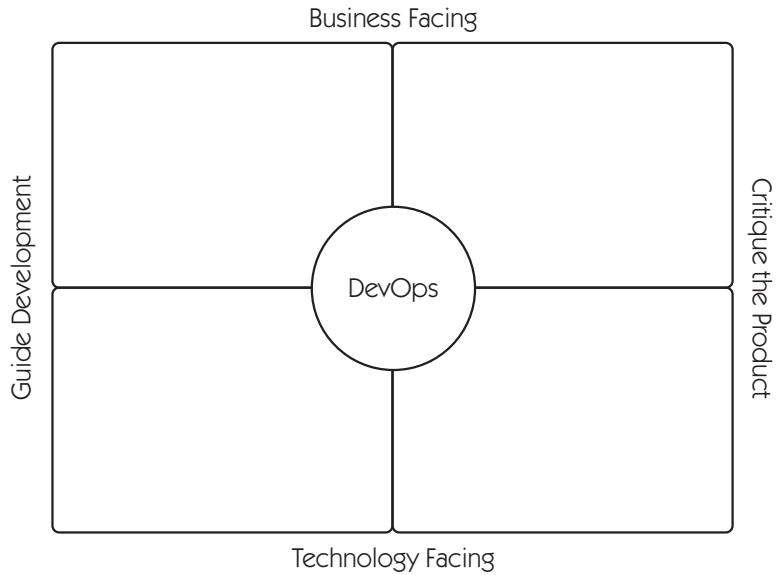


Figure 23-2 Test activities intersect at DevOps.

Lisa's Story

Back in the mainframe days when I worked as a programmer/analyst, I collaborated closely with the machine room operators. Together, we learned ways to solve and prevent problems with processing batch jobs.

In the early 1990s I was fortunate to learn UNIX system administration skills from experts on my team. I learned the importance of repeatable release processes and ways to verify the packaged product. In the 2000s, my teammates with in-depth system administration skills helped find better test automation frameworks and ways to configure CI jobs for quicker test feedback. They showed me how to use monitoring and log files to help with debugging and exploring.

My current team is the first I've worked with that uses the term *DevOps*. We have a backlog of DevOps user stories to improve all aspects of our infrastructure, including our test environments, CI, and deployment process. Interested team members join a weekly DevOps meeting to prioritize work, and these activities are planned along with product features. Programmers work with the company that hosts our production site and maintains our search engine software. We do our own production deploys and can respond quickly to operational problems. We experiment with ways to continually improve our delivery process.

Some companies maintain separate operations or IT departments. The members of these silos often have to support multiple teams, juggling development needs with production maintenance. They may lose touch with the development teams and their needs.



Lisa once facilitated a workshop for a large company where participants from teams working on different product areas identified issues that were impeding various aspects of their testing. Testers complained that the other parts of the system had incompatibilities with theirs, and they had no idea how to overcome these. One of the workshop participants was from the operations department. Once he saw the obstacles that were listed, he said, “I had no idea you were struggling with this. My team can take care of these problems. From now on, contact me directly!” Sometimes solving a thorny testing problem just takes getting people from different departments in the same room.

Seeing the Whole: Add Infrastructure to the Testing Scope, DevOps Style

Michael Hüttermann, author of *DevOps for Developers* (Hüttermann, 2012), shares a success story he had with a whole-team approach to supporting infrastructure and testing through DevOps.

In a bigger project with about 100 developers, we had to cope with aggressive time-to-market targets, protecting competitive advantages, a lot of technical complexity, as well as high demands on availability and capacity. These were the main drivers to implement the DevOps approach. This approach included applying “the infrastructure as code” paradigm, which starts with putting Puppet manifests to version control, in our case Git. [Authors’ note: See the “Tools” section of the bibliography for links to all tools mentioned in this chapter.]

An end-to-end, department-spanning delivery pipeline was in place, starting with stage 0, the developer workspace, and closing with higher test machines, a production mirror, and production. From the version control system, baselines were created continuously. Those baselines contained different configuration units that made up a release, including business code, unit tests, integration tests, and infrastructure information. Building on top of these baselines, we created release candidates continuously. Cherry-picked release candidates

were promoted to be released to production. Those releases were both fit for purpose (by containing the functional scope) and fit for use (by conforming to the nonfunctional requirements).

After we applied DevOps concepts, the processes and tools of both development and operations were aligned with each other. Each used the same approaches to provision machines, including installation and configuration of infrastructure, middleware, and the business application. Development and operations continually collaborated to ensure maximum knowledge and open information exchange. We had slack time to learn and experiment, and we built mutual respect. We used kanban to manage the flow and reviewed the design early in the process. This enabled early and frequent learning, and “failing fast.”

By applying DevOps practices, service-level requirements and service capabilities could be defined and checked early in the process. Developer machines and test machines were production-like, thus they delivered fast and meaningful feedback of installation and configuration, as well as nonfunctional requirements such as security and monitoring.

As part of the continuous delivery platform, Puppet was used together with Vagrant and Jenkins to set up and remove virtual machines for reproducible testing of the provisioning process itself, as well as the defined result on the target machine.

In order to find errors early and often, we started with checking for more basic failure categories. As part of a continuous build, the infrastructure code in the form of Puppet manifests was validated early in the process. When syntactical errors in the manifests were found immediately, the process was aborted and no binaries were produced for further usage. Finding this category of errors is easy to achieve by just applying a `puppet parser validate` as part of a dedicated build step in the continuous build.

After the actual configuration of the target test environment with Puppet, another downstream stage was added to check the correct provisioning. We introduced test manifests that were applied by `puppet apply --noop` to check for Puppet compilation errors, and then checked the resulting log of events. A basic automated smoke test compared actual and desired results.

When the delivery pipeline was mature enough, we added code-style verifications. We found it helpful to agree on a shared format for all our artifacts. Many checks on business code were done with SonarQube, but we also checked style guide compliance of infrastructure code with `puppet-lint`.

Applying DevOps practices taught us that it was crucial to foster the spirit of the “one team” that consists of members of development, coders, testers, business, and operations, along with network, systems, and database engineers. All involved experts became developers of the solution.

The combined team shared business goals such as reducing cycle time. Shared processes, such as using the same provisioning concept for all machines, and shared tools, such as Puppet, also aligned the combined team. Many production incidents are caused by changes to the IT infrastructure or by unplanned work, such as firefighting production incidents due to broken processes or badly tested solutions. Puppet allowed us to define executable specifications of target infrastructure behavior, making the infrastructure testable. This made automation reliable, which shortened feedback cycles. The documentation about the machines is always up-to-date, facilitating conversations between developers and operations staff.

It was elementary to distinguish between accountability and responsibility. Colleagues of the “one team” of devs and ops were collectively responsible, but only one person from the development department was accountable for development machines and one person from the operations department was accountable for operations machines. Other colleagues who were neither responsible nor accountable were consulted or informed, early and often.

Overall it was a great success to emphasize the entire flow of work from start (inception) to end (operation), to extend agile development testing practices to operations, and to intensively include operations staff starting with the early phases of software development.

To achieve the reliable test automation and shortened feedback cycles that Michael’s team enjoyed, you need to understand your build pipeline. Know what tests run in which environment. The concept of build pipelines works well for talking about test environments. Building good test environments has been an obstacle to many of the testers and teams with which Janet has worked. In Chapter 5, “Technical Awareness,” Figure 5-2 was an example of a very simple build pipeline. Generally speaking, there is an automated push to the development environment if the CI tests all pass. However, one of the practices that we recommend is to use a pull system for the test environment. This gives testers control over their test execution. In our opinion, there is nothing worse than to be halfway through an exploratory test cycle, only to have the build

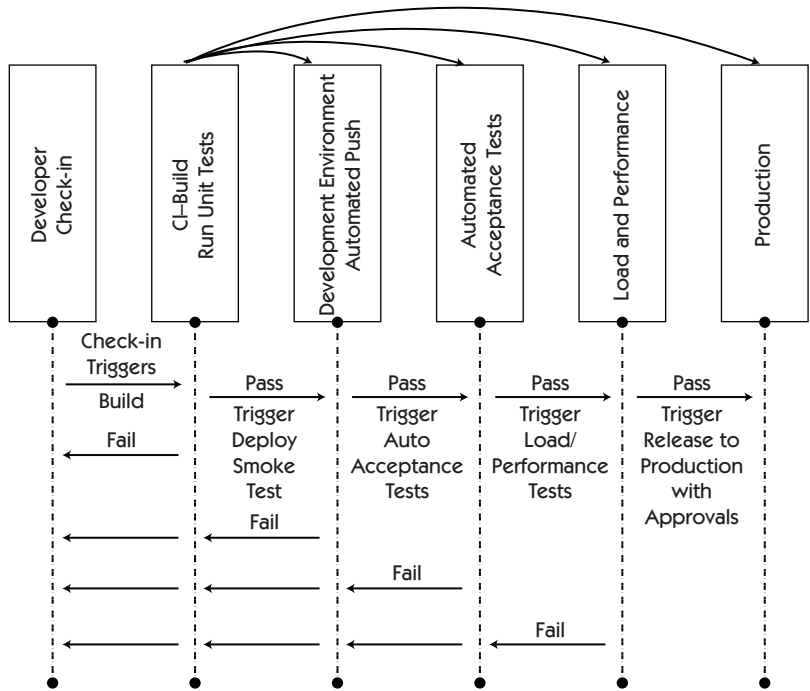


Figure 23-3 Automated build pipeline

you were testing overwritten. That often means starting the testing over from scratch.

Figure 23-3 is a diagram of an automated build pipeline. If you compare it to the simple build pipeline in Figure 5-2, you'll see that the test and the staging environments in this example are not part of the automation. Those are two environments where you would perform exploratory testing.

In Chapter 18, "Agile Testing in the Enterprise," we presented Dell's testing strategy for its enterprise solution. This next story gives more details on the infrastructure needed.

Automated Build Verification Testing

Kareem Fazal, a software engineer at Dell, talks more about their complex build verification and how they adapted it after their first attempts.

The Hardware Test-Matrix Challenge

One of the challenges faced in virtually all of our software test environments at Dell Enterprise Solutions Group (ESG) is the extensive hardware compatibility matrix that our software is expected to support. This is most evident in our Server Systems Management firmware and software projects supporting our twelfth-generation servers. When we first applied agile practices to our 12G servers, we established a CI and automated build verification test (aBVT) environment. This CI and aBVT environment supported a project with 15 Scrum teams distributed across India and the United States. Since new code was checked in several times an hour, there was a constant demand for frequent, “known-good” builds. The automated build and tests needed to finish within 90 minutes, running against multiple physical unit-under-test (UUT) environments that were configured with specific network interface card (NIC) and RAID combinations.

The procedures that we initially used for aBVT were not very flexible. It was difficult to add test suites that did not follow a predefined interface format to the existing suite. Also, since test cases were bound to servers, if a specific configuration was required that wasn’t already in place, a server had to be taken offline and reconfigured. This meant that we had to be very careful about what tests were run and how much time a test scenario took in order to balance resource use with testing requirements. Under these conditions, it was easier to simply reduce the number of tests run and limit each test run to only those test cases that matched the hardware configuration that was in place. To make matters worse, this was a single-threaded environment, so although there were multiple incoming build requests every hour, they were bottlenecked behind the single, in-progress build verification test run.

The Solution

In preparation for the next update release, the CI and aBVT team took steps to address the limitations of the test environment. The build team envisioned a solution that provided

- Scrum teams with “ease of use” in developing and submitting test cases to achieve maximum hardware coverage
- Build teams with “efficient use of hardware resources” without requiring manual intervention between test runs
- Scrum teams with the “flexibility” to prioritize test cases on specific configurations

In response, our team developed the generic resource manager (GRM). The GRM achieved the flexibility and ease of use we were seeking by having all configurations defined by a simple set of stand-alone XML files. The name space was defined in a way that did not place arbitrary limitations on what information could be defined, while remaining basically simple and easy to understand. The manner in which the configuration was used allowed for good scalability as well. The tester could define as much or as little information for each testing scenario as was required with very little superfluous effort.

The GRM achieved efficient use of server resources by making sure that they were used in parallel as much as possible, and it allocated resources based on the test requirements. If a test takes longer than expected or ends with an error, the GRM adapts to the conditions at hand. Resource requirements that are defined for a particular test can be as broad or as narrow as required without having an effect on other tests in the test run.

The GRM can be used for small-test setups, but it can also scale and support complex scenarios with many types of resources and thousands of tests in the mix. This is accomplished with a robust, easy-to-understand configuration methodology using a standard XML file format.

Results

With the deployment of the GRM into the aBVT environment, the teams were able to maximize test matrix coverage for each run of a test suite. Because test cases for a particular run did not always require, or consume, all available configurations, additional aBVT runs were initiated in parallel, to make maximum use of available configurations. The GRM provided the build team with the flexibility to scale up and add configurations and test cases without disabling any in-progress test runs, easing maintenance concerns.

As you can see from Kareem's story about his experiences, there is no "one size fits all." Dell ESG custom-built a flexible solution that worked for teams around the globe. Look at your own environments, and strive to simplify the build and deploy and test cycle so that it is consistent and maintainable.

HOW TESTERS ADD DEVOPS VALUE

One area of DevOps where testers can contribute their specialized skills is in helping to test the infrastructure. They can help identify places where infrastructure might break simply by asking questions like, "How do we test what happens if the network goes down?" Testers may not know all the answers, but they are good at asking questions.

Janet worked with a company whose infrastructure testing group worked closely with the individual feature teams, but on a more formal basis. They created an agreement with the teams that looked something like this.

The "Levels of Service" that infrastructure testers are prepared to provide are:

- **No involvement:** *If testing is required, it will be planned, executed, and monitored by a representative from a team appropriate to the activity (e.g., DBA, Network, Infrastructure).*
- **Guidance:** *Infrastructure testers will provide guidance to assist with planning an appropriate test. The test will be executed by a representative from a team appropriate to the activity (e.g., DBA, Network, Infrastructure). Testers may review test results and assist with risk assessment for a release/acceptance decision.*
- **Planning:** *Infrastructure testers will prepare a risk-based test plan that defines the testing approach and scope, lists tests to be conducted and acceptance criteria, and identifies risks associated with the project. This plan will be turned over to a representative from a team appropriate to the activity (e.g., DBA, Network, Infrastructure) for execution. Infrastructure testers may review test progress and results and assist with risk assessment for a release/acceptance decision.*
- **Infrastructure testing:** *Testers will prepare a risk-based test plan that defines the testing approach and scope, lists tests to be conducted and acceptance criteria, and identifies risks associated with the project. Infrastructure testers will execute the tests, providing feedback to*

the project team on execution status. Issues will be recorded, tracked, actioned, and escalated using the standard defect-tracking system. Infrastructure testers will present test results and risk assessment for a release/acceptance decision.

Those definitions are more formal than we'd normally use in agile teams. However, it shows how testing specialists can contribute in different situations, helping to manage risk and provide feedback about the infrastructure, including the test and production environments. Earlier in the chapter we learned how Michael Hüttermann's teams tested their infrastructure using open-source tools such as Puppet, Vagrant, and Jenkins. We'll look at more examples.

Testing Infrastructure

Stephan Kämper, a tester from Germany, explains more details about how he tests infrastructure.

In my context, the infrastructure is the extra hardware and software that you need to execute tests. It typically includes a CI system, including jobs the system executes; build tools like make, rake, or Maven; and the machines on which the CI runs. Whether or not the (wireless) network or firewall should also be considered part of your testing infrastructure depends on your own context.

In my current team (as of late 2013), we run automated checks against every single commit to the source control system, create new software artifacts (deployable build items) regularly, and run stand-alone integration tests against these artifacts on newly built virtual machines. These artifacts are put through a system integration test and finally brought onto the production system via the CI system.

The whole workflow, from a single commit to production, is orchestrated and supported by the CI system. Since we wrote the code to do all this, it seemed like a very good idea to also test it. The ability to go live with changes depends on that code.

What Part of the Infrastructure to Test

I do not suggest that you test your CI system as such, and neither do I recommend that you test your build system. However, I do think the code you write and feed to these systems should indeed be tested.

As an example, let's assume you're using Jenkins as the CI system and rake as the build tool. We use the Jenkins Job DSL/Plugin, which allows us to write Jenkins jobs as code, in contrast to the usual way of setting them up via the graphical user interface. That way we can have our Jenkins code under version control.

Since rake tasks are Ruby code, the tasks delegate the work to Ruby classes, modules, or methods, which in turn can be developed and tested like any other code. The same is certainly possible for other combinations of build systems and programming languages.

One Caveat

Some Jenkins jobs actually change the “world”—that is, they deploy software to production systems. This is hard to test in a laboratory environment since the whole point of a production system is, well, leaving the safety of a test environment.

But even in this case, you can parameterize the Jenkins jobs so that you can deploy to a test or staging system and production using the same script. That way, deploying to a test system is in fact the test for deploying to production. Note that even when this works nicely, the deployment to production may still be broken, since you might deploy to the wrong server.

Stephan emphasizes the value of testing automated deployments. Other types of infrastructure testing that testers might perform are often part of Quadrant 4 tests, including connectivity, reliability, failover, or backup and restores.

Teams are often called upon to test in production, monitoring site activity or performance. Pairing to monitor production log files has advantages similar to those of pair programming and testing. It is easy to concentrate on one thing and completely miss another, so a second pair of eyes is invaluable.

We've talked about what DevOps does for testing and quality and how testers can help with DevOps activities. In the final part of this section, we'll cover the intersection of both.